# An AI Based High-Precision Industrial Thermometer

Puni Aditya, Vivek K. Kumar, Jnanesh Sathisha Karmar, Burra Shreyas Goud, G.V.V. Sharma
*Department of Electrical Engineering*
*Indian Institute of Technology Hyderabad*
Kandi, India
gadepall@ee.iith.ac.in

***Abstract*—Accurate temperature measurement is critical in industrial process control. Platinum Resistance Temperature Detectors (RTDs), specifically the PT100, are widely utilized for their stability and linearity. However, traditional calibration requires expensive, specialized hardware. In this paper, we propose a cost-effective methodology for calibrating PT100 sensors using machine learning (ML) and computer vision. Our primary contribution is an Optical Character Recognition (OCR) based data acquisition pipeline that captures real-time analog thermometer readings and pairs them with sensor voltage outputs during a dynamic thermal cycle, eliminating the need for thermal chambers. Furthermore, we evaluate multiple ML techniques for non-linear calibration: physically-informed Least Squares (LSQ) regression, Stochastic Gradient Descent (SGD), and a Random Forest Regressor (RFR). Evaluated across a high-precision Wheatstone bridge with a 16-bit ADC, the physically-informed LSQ quadratic model achieves superior accuracy, yielding a Mean Squared Error (MSE) of 0.009198 and an $R^2$ of 0.999956, outperforming the generic regressors.**

***Index Terms*—PT100, Machine Learning, Optical Character Recognition, Temperature Measurement, Calibration, Edge Computing.**

## I. Introduction

Temperature measurement is a fundamental requirement in process control, automotive systems, and instrumentation. Among the various contact temperature sensors available, the Platinum Resistance Temperature Detector (RTD), commonly known as the PT100, is considered the standard for precision thermometry [1]. PT100 sensors are widely favored in industrial applications for their linearity and long-term stability.

To achieve high accuracy, recent literature has explored both hardware and software optimizations for the PT100. Samuk and Çakır (2023) [2] implemented a high-accuracy circuit using specialized differential amplifiers. While effective, the design relies on expensive analog components and lacks an investigation into contemporary machine learning (ML) correction methods. Alternatively, Li et al. (2024) [3] focused on the design of DC-balanced Wheatstone transducers for PT100 sensors. However, their work relied entirely on software simulations, leaving real-world hardware feasibility unaddressed.

From a data-driven perspective, Liu et al. (2023) [4] successfully utilized Artificial Neural Networks (ANN) to non-linearly calibrate PT100 readings, drastically reducing error margins. Despite these advances, existing research typically demands expensive hardware testing environments or specialized dynamic testing equipment for accurate dataset generation.

To address the gap between expensive hardware calibration and software simulations, this paper presents a novel framework for PT100 calibration. First, we propose an automated Optical Character Recognition (OCR) based dataset generation pipeline that synchronizes analog readings with sensor voltages, completely circumventing the need for expensive thermal calibration chambers. Second, we present a comparative study evaluating various ML models to identify the most robust algorithm for steady-state non-linearity correction.

## II. PT-100 Fundamentals

The PT100 is defined by a nominal resistance of $100\,\Omega$ at 0°C. Unlike thermocouples which generate a voltage, RTDs differ in that they operate by changing electrical resistance in direct proportion to temperature changes [5]. This relationship is inherently non-linear over wide ranges, though it is highly predictable. The resistance $R(t)$ at temperature $t$ is governed by the Callendar-Van Dusen equation [6]

$$R(t) = R_0(1 + At + Bt^2) \tag{1}$$

for temperatures $t > 0$°C, where $A$ and $B$ are standard coefficients. This quadratic physical relationship serves as the theoretical justification for using LSQ models in this work.

While PT100 sensors offer high stability, the resistance change is relatively small ($\approx 0.385\,\Omega/$°C). Consequently, accurate reading requires precise signal conditioning to convert resistance changes into measurable voltage levels [7]. This paper aims to optimize this conversion by analyzing different hardware front-ends and software regression models to find an optimal solution for high-precision, low-cost temperature sensing.

## III. Hardware Setup

Two main analog front-ends were tested to evaluate the trade-off between circuit complexity and measurement accuracy. Table IV lists the components required for both analog front-ends, detailing the specific parts used for high-accuracy and cost-effective implementations. The detailed hardware component list, alongside the pin-connection configurations

for the ADS1115 and LCD, are provided in Appendix A to facilitate reproducibility.

- *High-Precision Wheatstone Bridge with ADC:* This uses a Wheatstone bridge configuration coupled with an external 16-bit ADS1115 ADC [8]. This setup allows for differential measurement, significantly reducing noise and common-mode errors. The connection details for integrating the external 16-bit ADS1115 ADC are available in Table V. The complete circuit schematic for the high-precision Wheatstone bridge setup is illustrated in Fig. 2.
- *Simple Voltage Divider:* This uses a simple voltage divider utilizing the Arduino's internal 10-bit ADC. The complete circuit schematic for the simple voltage divider setup is illustrated in Fig. 3.

The Arduino-LCD connections are available in Table VI. The code for the Arduino, which reads the voltage across the PT-100 and displays it on the LCD, alongside flashing instructions, is provided in our repository[1].

## IV. Automated Training Using OCR

The dataset is collected by simulating a real-world thermal change (cooling cycle) to capture paired temperature readings. Since the training data is available only on an analog thermometer, instead of manually noting the temperature, we automate the data collection process using OCR based techniques. Appendix B has more information on the software execution.

### A. Experimental Setup

The practical requirements and experimental setup are listed below

1) *Hardware:*
   - The arduino setup with the code from the previous section flashed.
   - A phone connected to your computer.
   - (Optional) An NVIDIA GPU with CUDA installed. The script is pre-set to use `gpu=True` for much faster processing.
2) *Software:*
   - Python 3.x
   - The required Python libraries: `opencv-python`, `easyocr`, and `numpy`.

### B. Data Collection Methodology

1) Setup: Place both the PT100 probe and the reference thermometer inside an electric kettle, submerged in water.
2) Heating Phase: Turn on the kettle and heat the water to its boiling point.
3) Recording Phase:
   - Switch off the kettle at boiling point.
   - Start recording a video using the smartphone.
   - Ensure both the PT100 LCD display and reference thermometer are visible in the frame.
4) Cooling Phase: Continue recording the cooling process as the water temperature drops naturally.
5) Data Extraction: The recorded video acts as the raw dataset.
6) Extract frames and apply OCR to detect readings from both displays.

[1]Arduino Data Collection Code: https://github.com/gadepall/pt100/tree/main/codes/arduino/datacollection

## V. Machine Learning Models

### A. Model Explanations

The LSQ and SGD models obtained from the dataset derived in the previous section are available in Table I. The coefficients are determined through linear least squares regression.

TABLE I: Extracted Polynomial Coefficients for Forward and Inverse Regression Models. (Note: $X$ represents sensor voltage in Volts, and $Y$ represents Temperature in °C)

| Model | Equation | $a$ | $b$ | $c$ | Description |
|---|---|---|---|---|---|
| Inverse LSQ (No ADC) | $X = aY^2 + bY + c$ | −0.000014990 | 0.005589496 | 2.526066163 | Inverse Least Squares with voltage divider setup without ADC. |
| Inverse LSQ (ADC) | $X = aY^2 + bY + c$ | −0.000006744 | 0.004344843 | 0.056019368 | Inverse Least Squares with ADC setup and Wheatstone bridge. |
| Quadratic LSQ (ADC) | $Y = aX^2 + bX + c$ | 147.376208110 | 197.568472526 | −10.357480491 | Least Squares (Quadratic) for Wheatstone bridge with ADC. |
| SGD | $Y = aX^2 + bX + c$ | 190.226107678 | 173.380527679 | −7.072446221 | Stochastic Gradient Descent model fit. |
| Inverse SGD | $X = aY^2 + bY + c$ | −0.000005386 | 0.004183549 | 0.060452454 | Inverse Stochastic Gradient Descent model fit. |

1) **Least Squares (LSQ) and Inverse LSQ Regression:** Chosen as our physically-informed baseline. The Callendar-Van Dusen equation naturally dictates resistance (and analogously, voltage $X$) as a quadratic function of temperature ($Y$). We define this mathematically native orientation ($X = aY^2 + bY + c$) as the *Inverse LSQ* model. However, for real-time inference, the system must predict temperature from voltage. Thus, we also derive the *Forward LSQ* formulation ($Y = aX^2 + bX + c$). Both models use least-squares optimization to minimize the sum of squared residuals, ensuring a globally optimal fit for the sensor's physical non-linearity.
2) **Stochastic Gradient Descent (SGD) Regressor:** SGD is a common iterative optimization algorithm used to fit a linear regression model [9]. We include it to test whether a standard linear approximation can effectively map the sensor data without relying on the physical quadratic model. It also highlights the necessity of proper data normalization in ML pipelines.
3) **Random Forest Regressor (RFR):** RFR is a standard ensemble learning method based on decision trees [10]. It is included to evaluate if a generalized non-linear machine learning model can outperform the physically based LSQ approach. Because tree-based models rely on discrete spatial partitioning (step functions), we investigate their ability to smoothly interpolate continuous analog variables like temperature.

The models are obtained and implemented using Python and the scikit-learn [11] library. The implementation of these

models and the evaluation of Mean Squared Error (MSE) and R-squared ($R^2$) metrics are available in the repository[2].

## VI. Model Evaluation and Comparison

Table II summarizes the Mean Squared Error (MSE) and R-squared ($R^2$) scores for all evaluated models [12]. MSE was specifically selected to heavily penalize larger temperature prediction deviations—a critical safety requirement in industrial control systems. Conversely, the $R^2$ metric is utilized to measure the proportion of variance in the actual temperature that is predictably explained by the sensor's input voltage. A robust industrial thermometer must achieve an MSE approaching zero and an $R^2$ approaching 1.0. Based on these metrics, the LSQ (Quadratic) model for the Wheatstone bridge with ADC vastly outperforms the rest.

TABLE II: Low MSE and high $R^2$ is desirable

| **Model** | **MSE** | $R^2$ |
|---|---|---|
| 1. Inverse LSQ (Volt. Div w/o ADC) | 0.6612 | 0.9977 |
| 2. Inverse LSQ (Wheatstone w/ ADC) | 0.016221 | 0.999922 |
| 3. LSQ (Wheatstone w/ ADC) | 0.009198 | 0.999956 |
| 4. SGD Regressor (Wheatstone w/ ADC) | 0.030477 | 0.999854 |
| 5. Inverse SGD (Wheatstone w/ ADC) | 0.016221 | 0.999922 |
| 6. Random Forest (Wheatstone w/ ADC) | 1.161318 | 0.994451 |

Table III presents a small subset of the test data, showing the actual temperature values alongside the predictions from the Random Forest, Inverse LSQ, LSQ, SGD, and Inverse SGD Regressor models for the Wheatstone bridge setup with ADC. The corresponding graph is available in Fig. 1. We thus conclude that the LSQ model is best at predicting the temperature. Though the results are provided only for a small range of temperatures in Table III and Fig. 1 for brevity and visualization respectively, the results hold true for all values of practical interest.

TABLE III: Wheatstone Validation Data and Predictions

| **Volt** (V) | **Actual** (°C) | **Inv LSQ** (ADC) | **LSQ** (ADC) | **SGD** (ADC) | **Inv SGD** (ADC) | **RF** (ADC) |
|---|---|---|---|---|---|---|
| 0.2634 | 51.9 | 51.9136 | 51.9070 | 51.7938 | 51.9906 | 53.3456 |
| 0.3127 | 65.9 | 65.7972 | 65.8329 | 65.7442 | 65.8829 | 67.6248 |
| 0.2083 | 37.2 | 37.1962 | 37.1906 | 37.2964 | 37.1134 | 37.0563 |
| 0.4037 | 93.1 | 93.6290 | 93.4194 | 93.9231 | 93.2383 | 89.0505 |
| 0.2092 | 37.5 | 37.4305 | 37.4238 | 37.5239 | 37.3513 | 37.0563 |
| 0.2978 | 61.6 | 61.5231 | 61.5485 | 61.4304 | 61.6219 | 61.8901 |
| 0.2028 | 35.8 | 35.7687 | 35.7707 | 35.9127 | 35.6628 | 36.7501 |
| 0.2666 | 52.7 | 52.7931 | 52.7892 | 52.6712 | 52.8748 | 53.3456 |
| 0.3185 | 67.5 | 67.4803 | 67.5183 | 67.4462 | 67.5568 | 68.7954 |
| 0.2850 | 57.9 | 57.9067 | 57.9202 | 57.7921 | 58.0053 | 58.6497 |

[2]Model Implementation: https://github.com/gadepall/pt100/blob/main/codes/models/PT100_models.ipynb

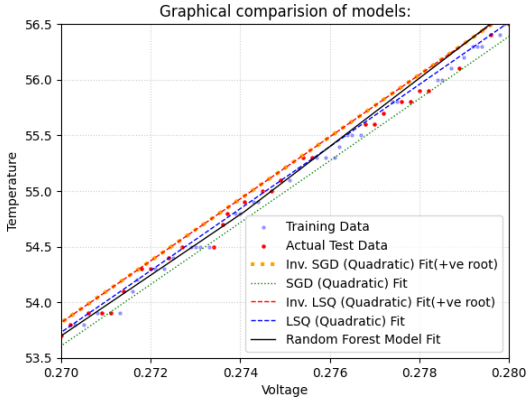


Fig. 1: Comparison of Random Forest, Inverse LSQ, LSQ, SGD and Inverse SGD Regressor predictions against actual test data for the Wheatstone bridge with ADC for temperatures between 53.5°C to 56.5°C

The code for the Arduino, which predicts the temperature from the voltage and flashing instructions are available in the repository[3].

## Appendix A
## Hardware Configuration and Schematics

TABLE IV: Component List

| **Component** | **Qty.** | **Description** |
|---|---|---|
| Arduino Uno | 1 | Microcontroller for processing and control. |
| ADS1115 ADC | 1 | 16-bit external ADC for high-precision voltage measurement. |
| PT100 RTD Sensor | 1 | Platinum resistance thermometer. |
| 100Ω Resistor | 1 | Precision resistor for Wheatstone bridge. |
| 4kΩ Resistors | 2 | Resistors for Wheatstone bridge. |
| JHD 162A Parallel LCD | 1 | For displaying the final temperature. |
| 22kΩ Potentiometer | 1 | To adjust the contrast of the LCD. |
| Breadboard | 1 | For creating solderless circuit connections. |
| Jumper Wires | Set | For connecting all the components. |
| Pen Thermometer <= 0.1°C resolution | 1 | Used for training purpose and as a reference. |
| Kettle/Heater with water | 1 | To heat the water, so that training data corresponding to a wide range of temperatures can be obtained |

TABLE V: ADS1115 Connections

| **ADS1115 Pin** | **Arduino Pin** | **Purpose** |
|---|---|---|
| VDD | 5V | Power Supply |
| GND | GND | Common Ground |
| SCL | A5 (SCL) | I2C Clock Line |
| SDA | A4 (SDA) | I2C Data Line |
| A0 | Input from Voltage Divider Node (between PT100 and R_REF) | Reading Voltage |
| A1 | Junction between the two 4kΩ resistors | Reference for differential reading |

[3]Temperature Prediction: https://github.com/gadepall/pt100/codes/arduino/inference

TABLE VI: LCD Connections

| LCD Pin | Arduino Pin |
|---|---|
| VSS | GND |
| VDD | 5V |
| V0 | Potentiometer Middle Pin |
| RS | Digital 12 |
| RW | GND |
| E | Digital 11 |
| D4 | Digital 5 |
| D5 | Digital 4 |
| D6 | Digital 3 |
| D7 | Digital 2 |
| A (Anode) | 5V |
| K (Cathode) | GND |

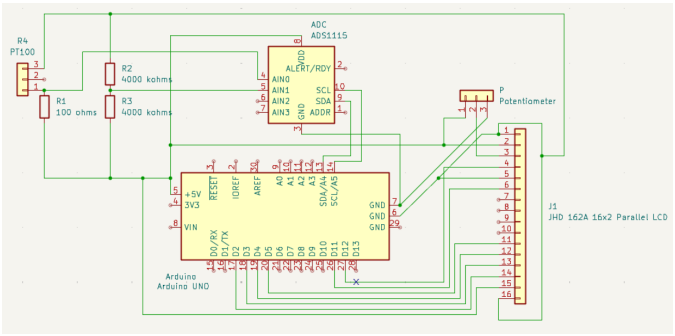


Fig. 2: Circuit Schematic of the High-Precision Wheatstone Bridge

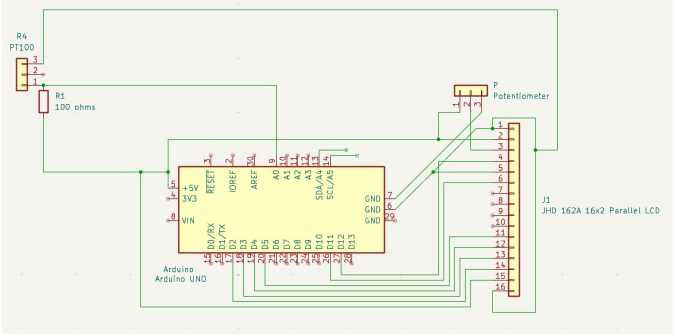


Fig. 3: Circuit Schematic of the simple voltage divider

## Appendix B
## Automated OCR Training Pipeline Setup

### A. Prerequisites

- Install DroidCam on your Android device.
- Install OBS Studio on your Linux laptop.
- Connect your Android phone to your Linux laptop.
- Open DroidCam on your phone. Open OBS studio on your laptop and slect DroidCam device from the menu.
- Install Python Libraries: The packages opencv-python, easyocr and numpy are required. Install the necessary packages using `pip`. Note: `easyocr` will also install `PyTorch [13]`, which is a large download and a required dependency.

### B. Collecting the Dataset

1) Go to the following directory https://github.com/gadepall/pt100/AutoTrainer/ and run the script `a.py`.
2) Two windows will appear: `'Voltage'` and `'Temperature'`. These show the exact (and processed) regions being sent to the OCR engine.
3) To stop the script, make sure one of these windows is active (click on it) and press the 'q' key.
4) This script uses OpenCV and EasyOCR [14] to capture video from a webcam, identify and read text from two specific regions (voltage and temperature in this case), and log this data to a file. It is designed to be calibrated for a specific, fixed-camera setup, such as this case where we are reading data from an LCD screen and a separate standard thermometer.

### C. Output

The script generates two types of output:

- `out.txt`: A text file that logs the detected readings. Every 15 frames, if text is found in both regions, a new line is added, formatted as: `[voltage_reading] [temperature_reading]`.
- `img/` Directory: This folder saves `voltageXX.png` and `tempXX.png` images. These are the exact frames that were sent to EasyOCR. If you are getting bad readings, check these images to see why. They are perfect for debugging your calibration and fixing the readings.
  The directory structure will look similar to this:

```
codes/
|-- out.txt
|-- img/
|   |--voltage_1.png
|   |--temperature_1.png
|   |--voltage_2.png
|   |--temperature_2.png
|   |--voltage_XX.png
|   |--temperature_XX.png
```

  `out.txt` may not be perfect due to some instances of incorrect character recognition. In such case, the data may have to be manually updated and saved in `trainingdata.txt`.
- The final training data is available in the following text file https://github.com/gadepall/pt100/blob/main/AutoTrainer/trainingdata.txt
  This was obtained from out.txt after manually modifying some of the incorrect entries.

### D. Important: Calibration is Required!

This script will not work correctly without calibration. Following are the adjustments that are necessary in `a.py`:

*1) GPU Usage:* If you do not have a compatible NVIDIA GPU, you must change this line

```
# From:
reader = easyocr.Reader(['en'], gpu=True)
```

```
# To:
reader = easyocr.Reader(['en'], gpu=False)
```

The script will run much slower on the CPU but will still function.

*2) Frame Cropping:* This is the most critical part. The script has to be told where exactly to look for the data. In the following, the parameters $h/2, w/3$ may need to be modified.

```
# Change the crop values according to requirement
voltage_frame = frame[0:int(h/2), :]
temp_frame = frame[int(h/2):, 0:int(w/3)]
```

In the above snippet, `voltage_frame` is currently set to the top half of the entire camera feed. `temp_frame` is set to the bottom-left third of the feed. These pixel/percentage values to need to be changed to precisely isolate the voltage and temperature readings. The `Voltage` and `Temperature` windows show what the script sees in these cropped zones, so the values may be adjusted accordingly and re-run until they are correct.

*3) Image Processing:* The following code snippet for reading the physical thermometer may have to be modified to fit the specific camera, lighting, and display setup.

```
# These values need to be calibrated according to
    requirement
gray_t = cv2.cvtColor(temp_frame, cv2.COLOR_BGR2GRAY)
t_lower = 50
t_upper = 100
_, thres = cv2.threshold(gray_t, 130, 255, cv2.
    THRESH_BINARY_INV)
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (11, 11))
closed_img = cv2.morphologyEx(thres, cv2.MORPH_CLOSE, kernel
    )
pretemp = cv2.bitwise_not(closed_img)
temp = cv2.GaussianBlur(pretemp, (17, 17), 0)
```

Table VII lists some of the values that need to be adjusted.

TABLE VII

| | |
|---|---|
| `cv2.threshold(gray_t,130, 255, cv2.THRESH_BINARY_INV)` | 130 is the threshold value. This is highly sensitive to lighting. Adjust it up or down until the text is clearly separated from the background in the `'Temperature'` window. |
| `cv2.getStructuringEleme nt( cv2.MORPH_RECT, (11,11))` | This is the kernel size for the `'closing'` operation, which helps connect broken parts of a number. You may need to make it larger or smaller. |
| `cv2.GaussianBlur(pretemp, (17,17), 0)` | This is the kernel size for the blur. |

We need to look at the `Temperature` window while calibrating. The goal is to make the numbers we want to read clear and solid, while removing as much background noise as possible. It is also preferred to hide the decimal point, either physically or by further image processing.

## References


[1] T. Bartelt, *Industrial Control Electronics: Devices, Systems & Applications*. Delmar Cengage Learning, 2005.

[2] A. Samuk and M. Çakır, "New circuit design and implementation for temperature measurement based on pt100 sensor," *ResearchGate*, 2023.

[3] M. Li, C. Zeng, and Y. Wang, "Pt100 temperature measurement sensors design and transducer simulation," *Journal of Physics: Conference Series*, vol. 2897, no. 1, p. 012011, 2024.

[4] C. Liu, C. Zhao, Y. Wang, and H. Wang, "Machine-learning-based calibration of temperature sensors," *Sensors*, vol. 23, no. 17, p. 7347, 2023.

[5] J. Fraden, *Handbook of Modern Sensors: Physics, Designs, and Applications*. Springer, 2016.

[6] H. L. Callendar, "On the practical measurement of temperature," *Philosophical Transactions of the Royal Society of London. A*, vol. 178, pp. 161–230, 1887.

[7] T. Mien and N. Hien, "Precision temperature measurement using wheatstone bridge," in *2019 International Conference on System Science and Engineering (ICSSE)*. IEEE, 2019, pp. 145–149.

[8] *ADS111x Ultra-Small, Low-Power, I2C-Compatible, 860-SPS, 16-Bit ADCs*, Texas Instruments, 2018, sBAS444D.

[9] H. Robbins and S. Monro, "A Stochastic Approximation Method," *The Annals of Mathematical Statistics*, vol. 22, pp. 400 – 407, 1951.

[10] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[11] F. e. a. Pedregosa, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[12] Y. M. e. a. Jaradat, "Beyond one-size-fits-all: Comparing and selecting regression metrics for robust model assessment," in *2025 12th International Conference on Information Technology (ICIT)*, 2025, pp. 416–422.

[13] A. e. a. Paszke, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035.

[14] JaidedAI, "Easyocr: Ready-to-use ocr with 80+ supported languages," 2020.